SatsPath

A transport-neutral Bitcoin payment discovery and routing protocol.

DISCLAIMER: This is hackathon software. Do not use with real funds.

Problem

Bitcoin has multiple payment rails — Lightning, on-chain, Ark, LNURL, BOLT12, Silent Payments — each with different trade-offs on speed, cost, and privacy. Users and apps must manually decide which rail to use, and there is no universal way to discover payment methods from a human-readable identifier.

Solution

SatsPath maps human-readable identifiers (e.g. `alice@example.com`) to **signed payment profiles** containing all available payment methods. A routing engine selects the best rail automatically based on amount, on-chain fees, and available methods.

Identity keys generated by SatsPath are used exclusively for signing profiles and proving ownership. **SatsPath does NOT generate wallet private keys, seeds, or custody keys for funds.** SatsPath is a resolution and routing layer — not a wallet, not a custodian.

SatsPath is specified as a transport-neutral protocol. Pear/Holepunch P2P is one optional transport for signed profiles, not the protocol itself. Start with docs/protocol.md, docs/resolvers.md, docs/wire_p2p.md, and docs/implementations.md.

The core flow is:

```
identifier -> resolver chain -> signed profile -> signature check -> route decision -> QR / wallet hando
```

Resolvers can use local files, HTTPS well-known endpoints, BIP-353 DNS, Nostr/NIP-05, or optional P2P. A receiver publishes the same signed SatsPath profile over any of those transports. A sender only trusts it after SatsPath verifies the profile signature.

Current Functional Surface

- Resolve `alice@example.com` to a signed payment profile through the resolver chain.
- Encode/decode universal payment URIs (`satspath:v1:<base64url_json>`).
- Sign profiles with secp256k1 identity keys.
- Verify profile signatures before routing.
- Resolve LNURL-pay endpoints and fetch BOLT11 invoices securely.
- Select the best payment rail (Lightning → On-chain → Ark preview/intents).
- Return a stable `QuoteResponse` with `ok`, `not_registered`, `no_route`, or `invalid_signature`.
- Produce a public payment payload / QR via CLI preview or `satspathd` wallet handoff.
- Fetch real-time on-chain fee estimates from mempool.space.
- Generate safe payment previews and Ark route intents; Ark swaps are not claimed as complete unless explicitly testnet-executed and supported by the bridge.
- Resolve DNSSEC-backed **BIP-353** payment instructions (`user@domain`) for domains the receiver controls — see docs/bip353_dns_resolution.md.
- Resolve SatsPath profiles over **Nostr/NIP-05** without P2P: NIP-05 discovers a Nostr pubkey and relays, then relays are queried for a signed SatsPath profile event.
- Generate invite links for unregistered users (receiver generates their own keys).
- Run a local daemon (`satspathd`) that exposes `/v1/node`, `/v1/quote`, `/v1/pay`, `/v1/dns/resolve`, `/v1/peers`, and `/v1/connections`.

- Full CLI: `init`, `register`, `show`, `encode`, `decode`, `quote`, `pay`, `invite`, `dns resolve`, `demo`.

  SatsPath can resolve DNSSEC-backed BIP-353 payment instructions for domains the receiver controls. For consumer email addresses like `gmail.com`, SatsPath uses platform verification / the invite flow instead, because the user cannot publish DNS records under `gmail.com`. BIP-353 resolution is **mainnet preview only**: SatsPath resolves and displays instructions but never pays, signs, or broadcasts.

Resolver Chain

The default resolver chain is:

```
local registry -> BIP-353 -> HTTPS well-known -> Nostr/NIP-05
```

Optional Pear/Holepunch P2P can be used as another transport, but the protocol does not depend on it.

For Nostr, publish:

1. A NIP-05 record at `https://<domain>/.well-known/nostr.json?name=<user>` that maps `user` to a Nostr pubkey and relay list.
2. A Nostr kind `30078` event by that pubkey with tag:

```
d = satspath-profile:<user@domain>
```

The event `content` is the JSON `SignedPaymentProfile`. Senders then resolve the identifier, verify the SatsPath profile signature, route, and get the QR/payment payload.

Consumer email domains such as `gmail.com` cannot be used for DNS or NIP-05 publication unless the user controls the domain endpoint. For those identifiers, the current behavior is invite flow or a future platform-verification resolver.

Email identifiers vs verified identifiers

SatsPath can use an email-shaped value as the human-readable identifier. That does not automatically verify the inbox.

- A signed profile proves control of the SatsPath protocol identity key.
- DNSSEC/BIP-353 or Nostr/NIP-05 can prove domain-controlled publication.
- A consumer email address such as `user@gmail.com` needs a platform challenge flow before it can be marked as an identifier-verified profile.
- Until that exists, consumer email addresses are treated as identifiers for lookup and invite delivery, not as verified payment identities.

Mainnet Preview

```
satspath preview rodrigo@satspath.dev 1000 --mainnet
satspath preview rodrigo@satspath.dev 1000 --mainnet --json
satspath quote rodrigo@satspath.dev 1000 --mainnet-preview --json
```

Lightning Address / LNURL preview does not fetch a real BOLT11 invoice by default. To fetch one explicitly:

```
satspath preview rodrigo@getalby.com 1000 --mainnet --fetch-lnurl-invoice
```

Fetched invoices are displayed only as data. SatsPath never pays them.

Nostr profile discovery

SatsPath can resolve a signed profile through Nostr without using the P2P SDK. This requires an identifier whose domain can publish NIP-05 metadata.

Receiver side:

1. Publish NIP-05 metadata:

`https://example.com/.well-known/nostr.json?name=alice`

with:

```
{
  "names": {
    "alice": "<64-char nostr pubkey hex>"
  },
  "relays": {
    "<64-char nostr pubkey hex>": ["wss://relay.example.com"]
  }
}
```

2. Publish a Nostr event authored by that pubkey:

```
{
  "kind": 30078,
  "tags": [["d", "satspath-profile:alice@example.com"]],
  "content": "{\"profile\":{...},\"signature\":\"3044...\"}"
}
```

`content` is the exact JSON `SignedPaymentProfile` exported by SatsPath.

Sender side:

```
satspath quote alice@example.com 250 --mainnet-preview --json
```

The resolver chain discovers the Nostr pubkey/relays through NIP-05, fetches the SatsPath profile event, verifies the SatsPath signature, and returns the normal `QuoteResponse` with a QR/payment payload.

For `rodrigodiazgt7@gmail.com`, neither DNS nor NIP-05 can be published by the user under `gmail.com`. That identifier currently resolves through invite flow unless a future platform resolver verifies Gmail ownership and publishes a signed profile through a supported transport.

What this prototype does NOT do yet

- Execute real Lightning, on-chain, or Ark payments. The experimental swap engine is **testnet-only scaffolding that previews swap intent**; it does not complete, direct, claim, or refund swaps, and must never be used on mainnet.
- Enable mainnet Ark execution.
- Verify email or domain ownership during registration.
- Publish profiles to Nostr from the Rust CLI. The resolver can read Nostr events, but publishing still needs a Nostr client or a future CLI command.
- Provide a decentralized global registry. SatsPath uses resolver transports; it does not have a single canonical registry.
- Support key rotation or profile revocation.
- Implement BOLT12 invoice fetching.
- Support Silent Payments or Split Payments.
- Provide a production wallet UI.

Safe Path (Mainnet Preview) vs Experimental Execution

The **Safe Path** (Mainnet Preview) is active by default and allowed:

- Resolving public data only.
- Fetching and resolving LNURL-pay metadata.
- Requesting BOLT11 invoices from LNURL callbacks securely.
- Presenting the invoice as a QR code or payment pointer for the user to pay with their own wallet.

- Profile signature and expiry checks.
- Public ownership proof checks where available.
- BIP21 on-chain URI generation.
- Ark public pointer / intent URI generation.

The **Experimental Path** (Automatic Execution via Swaps/Ark) is isolated and restricted:

- Requires explicit `--experimental-swaps` and `--testnet` flags.
- Mainnet execution is strictly disabled (no `--execute-mainnet` flag exists).
- No automatic payment execution exists on mainnet.
- No transaction signing or broadcasting path is exposed.
- No seed phrases, xprv/tprv, macaroons, certs, API secrets, claim keys, or refund keys are accepted into mainnet preview output.

Architecture

```
Human-readable identifier
  (e.g. rodrigo@satspath.dev)
          |
          v

    Resolver /          <-- local .satspath/registry.json (prototype)
    Transport chain     <-- BIP-353 / HTTPS / Nostr / optional P2P

           |
           v

    Signed Payment
    Profile
    (secp256k1 sig)

           |
           v

    Route Engine

        |    |    \
        v    v     v

     LN    BTC     Ark
```

Crate layout

```
satspath/
  crates/
     satspath-core/     # Types, crypto, codec, registry
     satspath-router/   # Route selection engine + fee API
     satspath-cli/      # CLI binary
  examples/
     rodrigo_profile.json
     demo_flow.md
  docs/
      architecture.md
```

```
implementations.md
resolvers.md
threat_model.md
protocol.md
wire_p2p.md
```

Installation

```
git clone https://github.com/<your-org>/satspath
cd satspath
cargo build --release
# Binary at: target/release/satspath
```

Or run directly:

```
cargo run -p satspath-cli -- <command>
```

CLI Usage

Initialize

```
satspath init
```

Creates `.satspath/registry.json` and `.satspath/keys.json` locally. These directories are git-ignored.

Register

```
satspath register rodrigo@satspath.dev
```

Generates a fresh secp256k1 identity keypair, builds a demo payment profile with Lightning, on-chain (×2 for privacy), and Ark methods, signs the profile, and stores it in the local registry.

Show

```
satspath show rodrigo@satspath.dev
```

```
Alias:          rodrigo@satspath.dev
Identity pubkey:02a1b2c3...
Fingerprint:    a1b2c3d4
Signature valid: yes
Updated at:     1735000000

Methods:
  - Lightning Address [Lightning]
      Lightning Address: rodrigo@satspath.dev
  - Bitcoin (primary) [On-chain]
      Address: bc1q...
  - Bitcoin (secondary) [On-chain]
      Address: bc1q...
  - Ark [Ark]
      Server: ark.satspath.dev
```

Encode a payment URI

```
satspath encode rodrigo@satspath.dev 21000 --memo "coffee"
```

```
satspath:v1:eyJ2ZXJzaW9uIjox...
```

Decode a payment URI

```
satspath decode "satspath:v1:eyJ2ZXJzaW9uIjox..."
```

```
Decoded payment request:
  Version:     1
  Alias:       rodrigo@satspath.dev
  Amount:      21000 sats
  Memo:        coffee
  Profile hint:(none)
```

Quote (route selection)

```
satspath quote rodrigo@satspath.dev 21000
```

```
Resolving alias...
Verifying signature...
Checking payment rails for 21000 sats...

Route Quote:
  Selected rail:   Lightning
  Label:           Lightning Address
  Reason:          Amount (21000 sats) is below 100000 sats threshold and Lightning is available.
  Estimated fee:   2 sats
  Confirmation:    instant
```

Pay (simulated)

```
satspath pay rodrigo@satspath.dev 21000 --experimental-swaps --testnet
```

```
Resolving alias 'rodrigo@satspath.dev'...
Verifying signed profile...
Checking available payment rails...
Selected route: Lightning
Reason:         Amount (21000 sats) is below 100000 sats threshold and Lightning is available.

Payment status: simulated_success (or real execution if experimental engine is enabled)
```

Invite (unregistered user)

```
satspath invite julian@example.com 21000
```

```
'julian@example.com' is not registered on SatsPath.

Invite link:
https://satspath.local/claim?alias_hash=a1b2c3d4...&amount=21000

WARNING: The receiver must claim this payment by generating their own keys locally.
```

Full demo

```
satspath demo
```

Runs all steps above automatically.

Example demo flow

See examples/demo_flow.md for a full annotated walkthrough.

Security model

- **Keys are local.** The only private keys SatsPath generates are **identity keys used to sign public payment profiles** — they are not wallet spending keys, seeds, or custody keys, and they do not control funds. They are stored only on the user's machine in `.satspath/keys.json`, which is git-ignored and never transmitted.
- **Signatures are mandatory.** Every payment profile is signed with the owner's identity key. Signature verification happens before routing.
- **Public profiles only.** The registry stores only public data: pubkeys, addresses, and signatures.
- **Invite, don't proxy.** For unknown users, SatsPath creates an invite. The receiver generates their own keys.

Threat model summary

Threat	MVP Mitigation
Fake alias registration	Signature-bound profiles; first-come-first-served
Server tampering	secp256k1 signature verification on every resolve
Key replacement attack	Signature covers identity_pubkey; swap breaks sig
Profile replay	`expires_at` enforced locally and remotely
Email takeover	Not mitigated at MVP (future: DKIM challenge)
LNURL spoofing	Amount and expiry verification before routing
On-chain privacy leaks	Multiple on-chain addresses per profile
Lost keys	Local backup only (future: BIP-39 seed phrase)
Malicious invite links	Alias hash + amount in invite; receiver verifies
Ark server trust	Mock client only (future: covenant-based Ark)

Full threat model: docs/threat_model.md

Future integrations

Integration	Purpose
BIP-353	Replace local registry with DNS TXT record resolution
Nostr	Decentralized identity and profile discovery (NIP-05, NIP-57)
Lightning Address	Auto-discover LNURL-pay endpoint from `user@domain`
BOLT12	Async invoices and reusable payment offers

Integration	Purpose
Ark	Non-custodial off-chain payments via virtual UTXOs
Silent Payments	Privacy-preserving on-chain payments (BIP-352)
Split Payments	Route a single payment across multiple rails

Running tests

```
cargo test
```

Test coverage includes: - Profile serialization - Profile signing and verification - Invalid signature rejection - Tampered profile rejection - Encode/decode payment URIs - Registry register/resolve - Router: Lightning for small amounts - Router: On-chain for low fees - Router: Ark fallback for high fees - Ark route planning and testnet-gated intent safety - Router: Error when no methods available - Registry persistence across opens

---

Hackathon scope

This prototype was built during a hackathon to demonstrate:

1. A working secp256k1-signed payment profile format with expiration.
2. A universal URI encoding scheme for payment requests.
3. An automatic payment rail selector using real mempool.space fee data.
4. A privacy-aware profile with multiple on-chain addresses.
5. An invite flow that preserves receiver key sovereignty.
6. A fail-closed, encrypted swap storage engine (behind `--experimental-swaps`).

It is **not production-ready**. The registry is local, payments are simulated (unless experimental flags are used), and domain verification is not implemented.

DISCLAIMER

This is hackathon software. Do not use with real funds.